M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_10

10. Devtools Pages

Matt Frisbie[1]

(1) California, CA, USA

Modern browsers offer developers a rich suite off developer tools for debugging, profiling, and inspecting web pages. Browser extensions are able to supplement these tools by providing custom interfaces that live inside the developer tools interface and can access special developer tools APIs.

Web developers are the intended audience for devtools pages, so consumer facing extensions will likely not need to make use of them. In general, browser extensions will only need devtools pages if they wish to use the Devtools API to inspect or profile the web page.

> **Note** There are two separate concepts in this chapter with similar names: “devtools pages” and “developer tools”. The term “developer tools” refers to the browser’s native interface that is opened by right clicking on a web page and selecting “Inspect”. The term “devtools pages” are extra interfaces added inside the developer tools. Browser extensions use the Devtools API to add these extra interfaces.

Introduction to Devtools Pages

The browser’s developer tools is a special interface that is granted total control and transparency over the active web page (Figure 10-1). It is fully capable of inspecting and managing the HTML, adding and modifying the page, and sniffing all network traffic. It is not restricted by cross origin rules, and so it is totally capable of inspecting web pages frames from any origin. Its

JavaScript debugger can pause execution and lay bare the entire memory model of a JavaScript runtime.

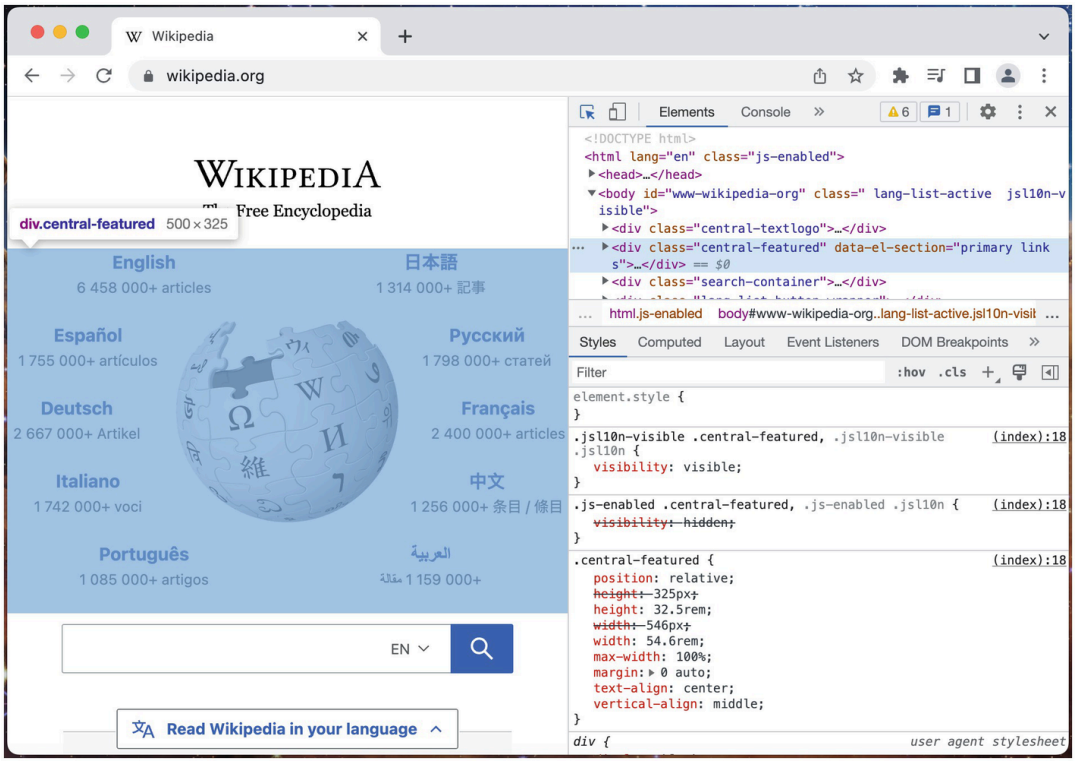

Figure 10-1 wikipedia.org with the Google Chrome developer tools opened to the Elements tab

The developer tools interface is complicated and information dense, so it relies on a hierarchical tabbed interface to break up these views into discrete pieces. Browser extensions can augment this developer tools interface by adding additional tabs. These added tabs are like any other extension-controlled user interface.

Creating a devtools Page

Browser extensions use an unusual strategy for adding extra interfaces to the browser's developer tools. A browser extension has the option to define the devtools_page property inside the manifest. The HTML page referenced will render as a headless page each time the browser's developer tools is opened. This headless page has a single purpose: it will load and execute scripts that use the Devtools API to insert additional interfaces.

The names can be confusing, so it's best to learn by example. Here is a simple extension that provides a devtools page that does nothing:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "devtools_page": "devtools.html"
}
```

Example 10-1a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <script src="devtools.js"></script>
  </body>
</html>
```

Example 10-1b devtools.html

```
// Use this script to access the Devtools API
```

Example 10-1c devtools.js

Don't worry about loading this extension, it doesn't do anything yet.

> **Note** The headless devtools page only runs each time the developer tools opens. This means that, when you make changes to the added panels, they will not re-render even if the extension is updated or the page is reloaded. When making updates to devtools pages, you must close and reopen the devtools interface or reload the updated devtools frames.

Adding panels and sidebars

Browser extensions can add two types of interfaces to the developer tools: panels and sidebars. They appear in different places inside the developer tools, but otherwise they behave identically. Both panels and sidebars have access to the same limited subset of the WebExtensions API as content scripts. Additionally, they are allowed to access the Devtools API.

> **Note** Most browser extension interfaces like popups and options pages are created declaratively: the manifest specifies an HTML file path as an entrypoint, and the browser understands how to interpret that and automatically incorporate the new interface. Conversely, devtools pages are created imperatively: to create an interface, you must call methods from the Devtools API and pass in an HTML file path.

Adding a Panel

To add a new panel to the developer tools, use the `chrome.devtools.panels.create()` method. Modify the previous example as follows:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>I'm the foo panel!</h1>
  </body>
</html>
```

Example 10-2a foo_panel.html

```
chrome.devtools.panels.create("Demo Devtools",
                              "",
                              "foo_panel.html");
```

Example 10-2b devtools.js

The `create()` method takes a title, an icon URL (here it is left blank), and a panel HTML file. With this extension loaded, close and reopen the developer tools and look to your panel menu. You will notice there is a new option called "Demo Devtools" (Figure **10-2**).

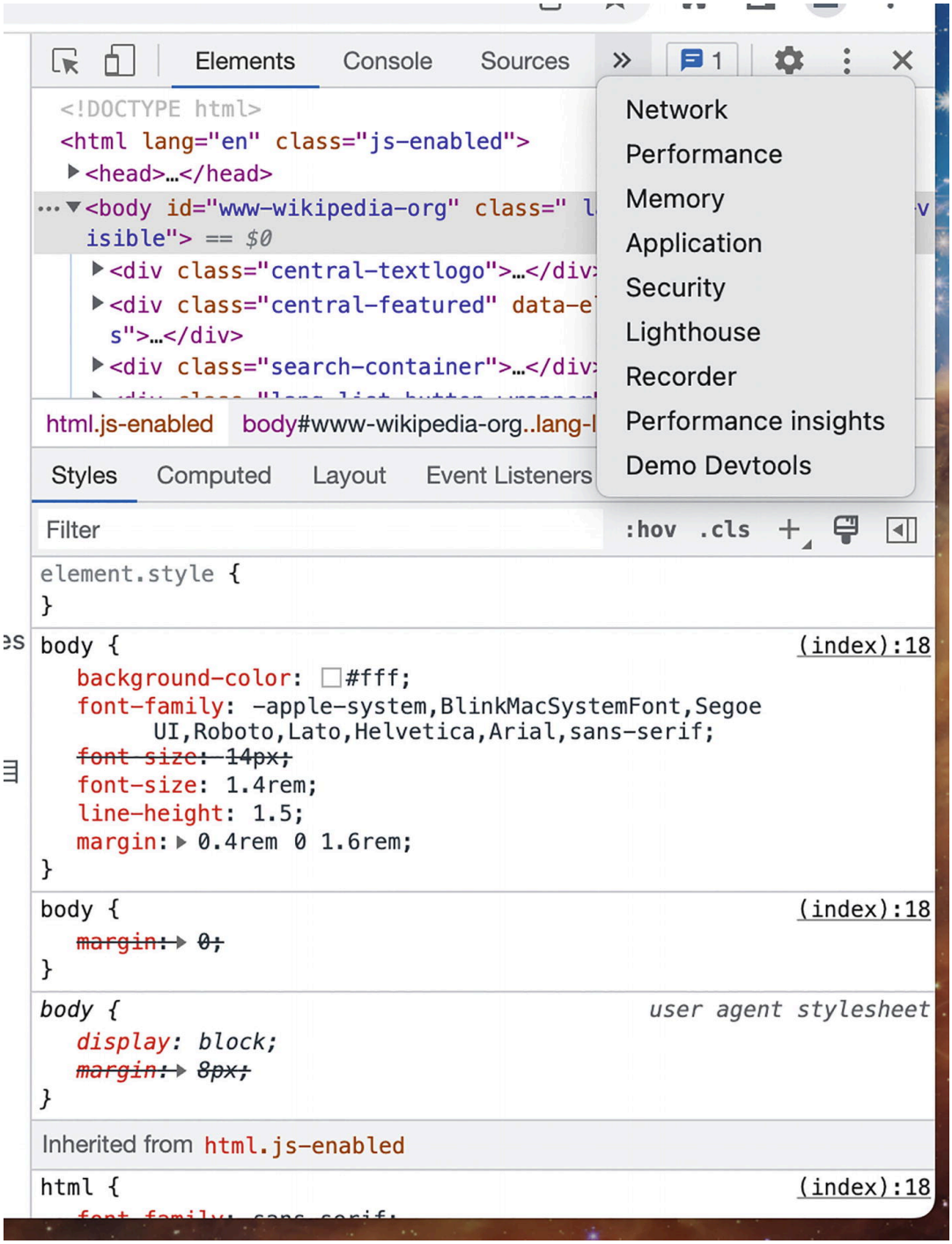

Figure 10-2 The added devtools panel

Click this option to reveal the new panel (Figure 10-3).

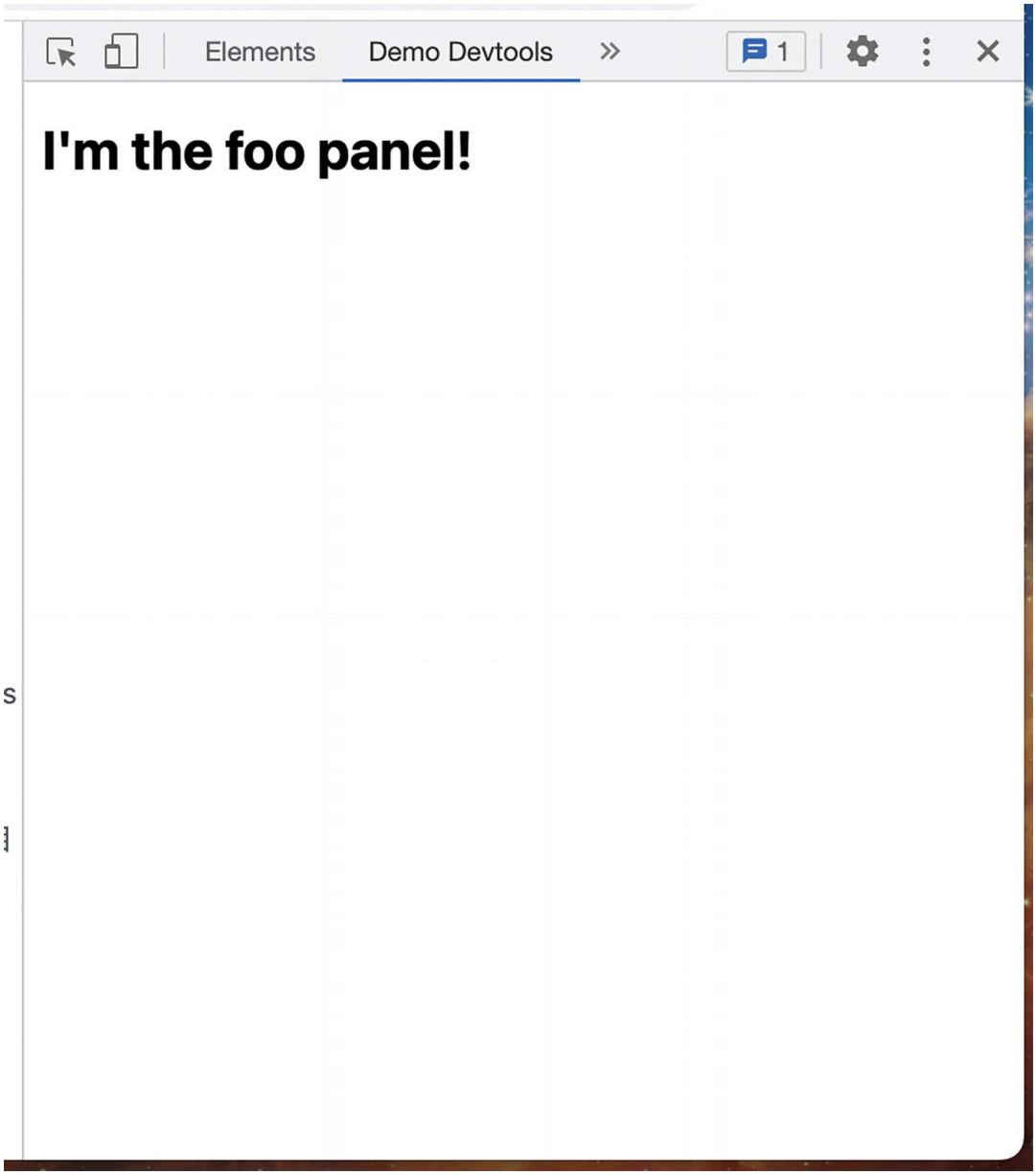

Figure 10-3 The newly added Demo Devtools panel revealed

Adding a Sidebar

Sidebars are slightly different from panels. Whereas a panel is a top-level interface in the developer tools, sidebars are interfaces that live alongside one of several existing interfaces inside devtools. Currently, there are two interfaces that you can add sidebars to:

- Elements
- Sources

Adding a sidebar is somewhat convoluted. You must call `createSidebarPane()` on the corresponding property for that interface: `elements` or `sources`. The callback for this method is passed the new sidebar pane as an argument, which you can use to add your custom page via the `setPage()` method.

The following example modifies the previous extension to instead add two sidebars: one to the elements interface, and one to the sources interface:

```
chrome.devtools.panels.sources.createSidebarPane(
  "Demo Sources Sidebar",
  (sidebar) => {
    sidebar.setPage("sources_sidebar.html");
  }
);
chrome.devtools.panels.elements.createSidebarPane(
  "Demo Elements Sidebar",
  (sidebar) => {
    sidebar.setPage("elements_sidebar.html");
  }
);
```

Example 10-3a devtools.js

```
<!DOCTYPE html>
<html>
  <body>
    <h1>I'm the elements sidebar!</h1>
  </body>
</html>
```

Example 10-3b elements_sidebar.html

```
<!DOCTYPE html>
<html>
  <body>
    <h1>I'm the sources sidebar!</h1>
  </body>
</html>
```

Example 10-3c sources_sidebar.html

Install this extension, and reopen the developer tools. In the *Elements* and *Sources* tabs, you will see a new child option (Figures **10-4**, **10-5**, **10-6**, and **10-7**).

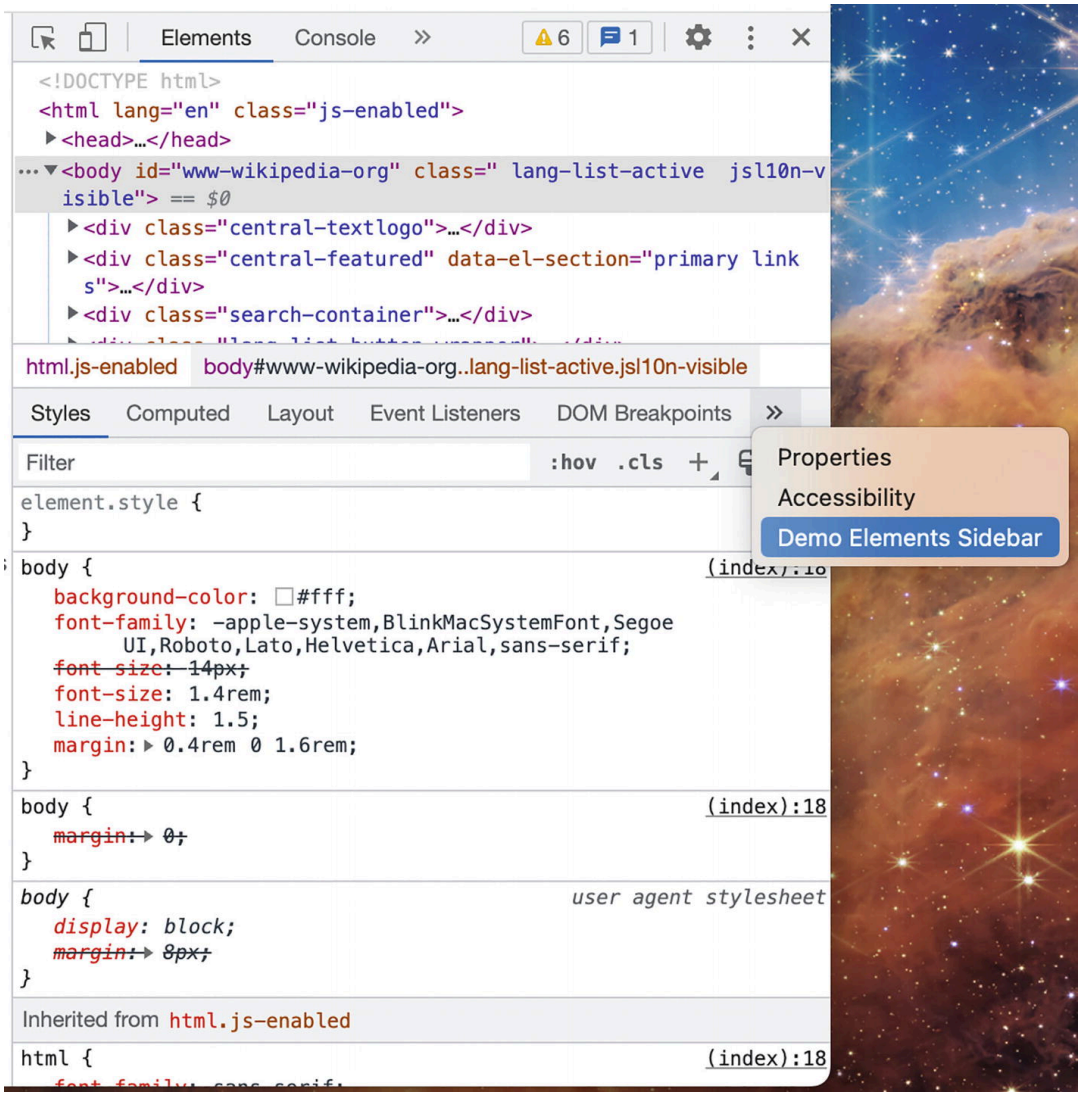

Figure 10-4 Revealing the new elements sidebar option

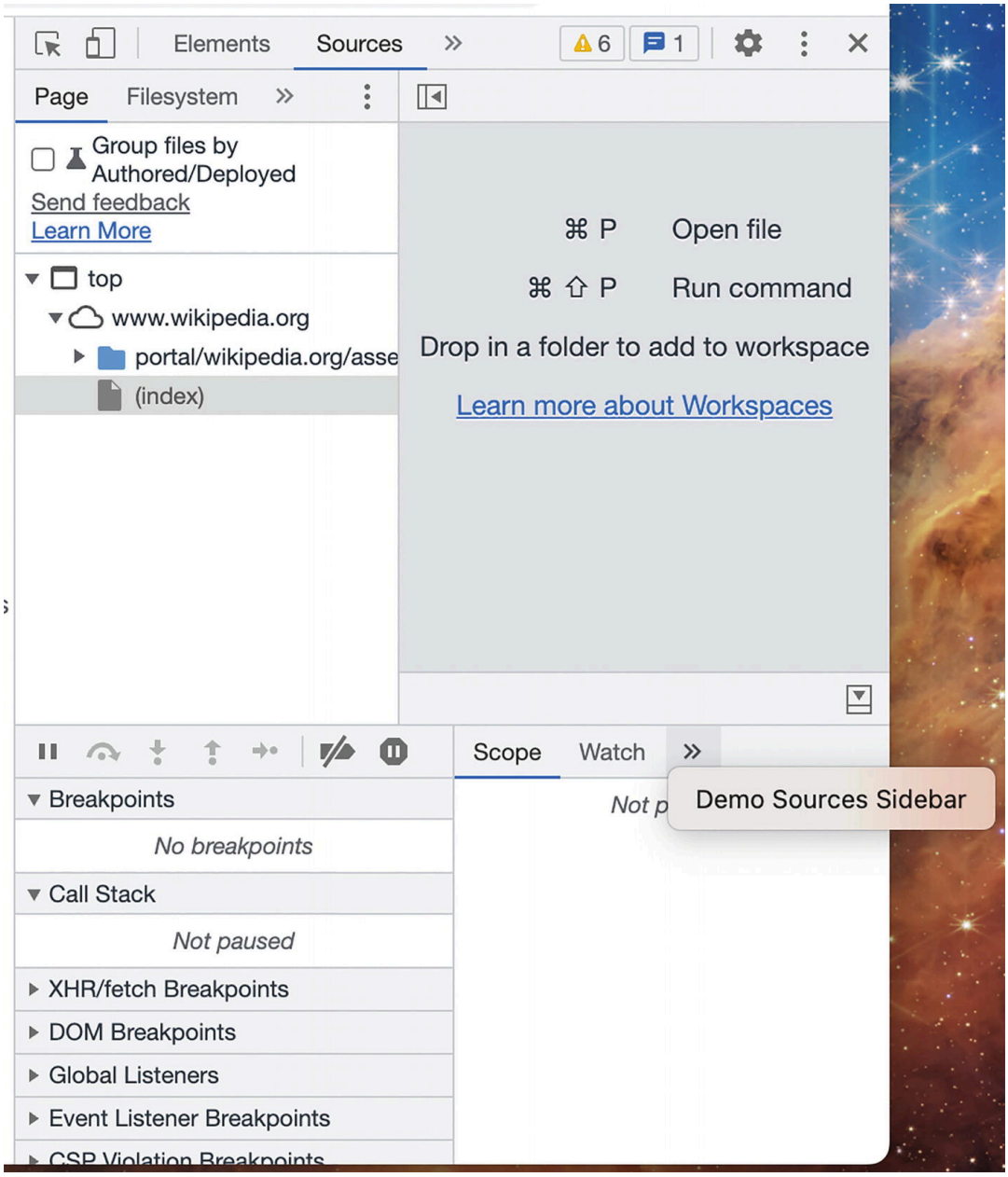

Figure 10-5 Revealing the new sources sidebar option

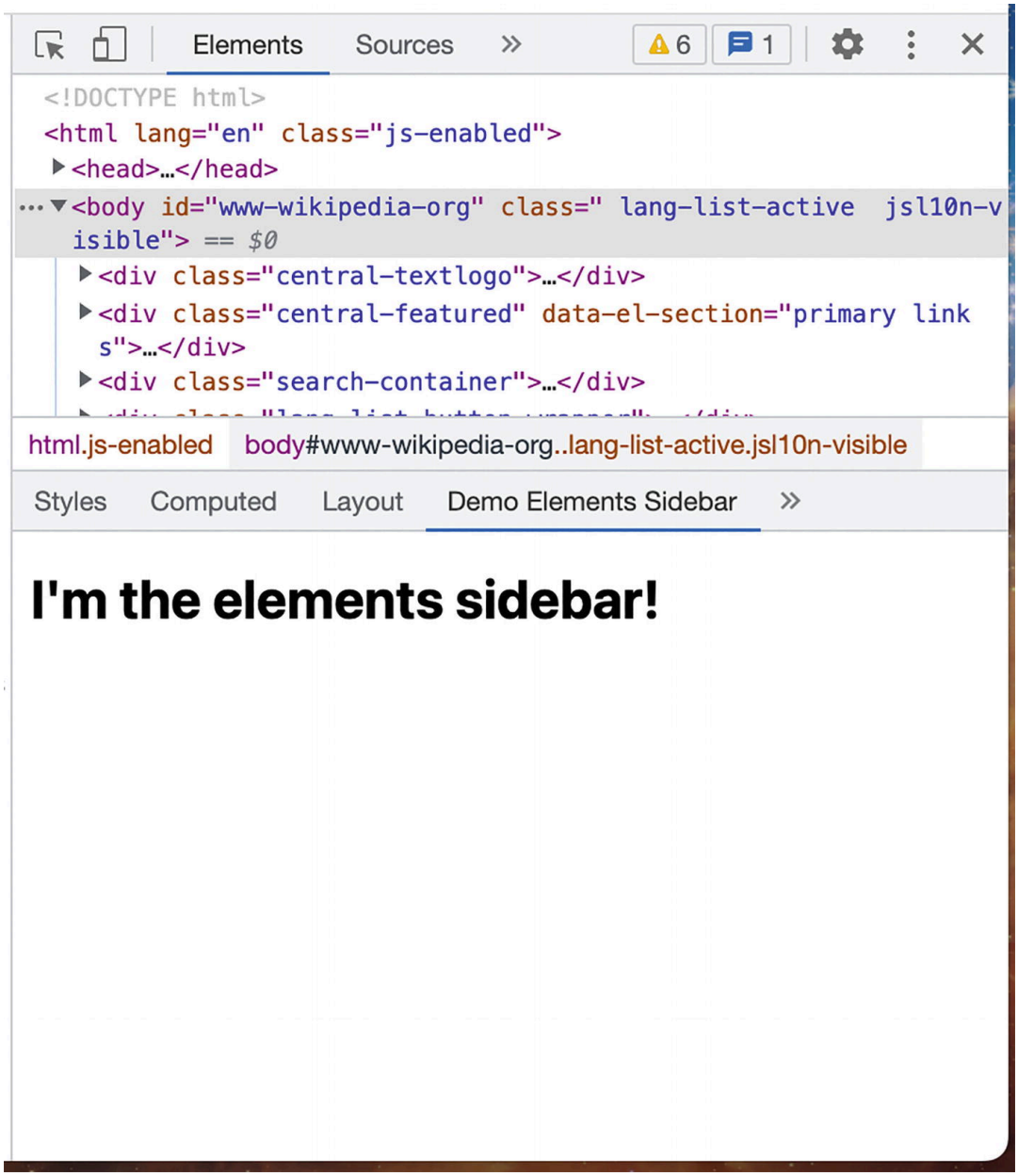

Figure 10-6 Viewing the new elements sidebar option

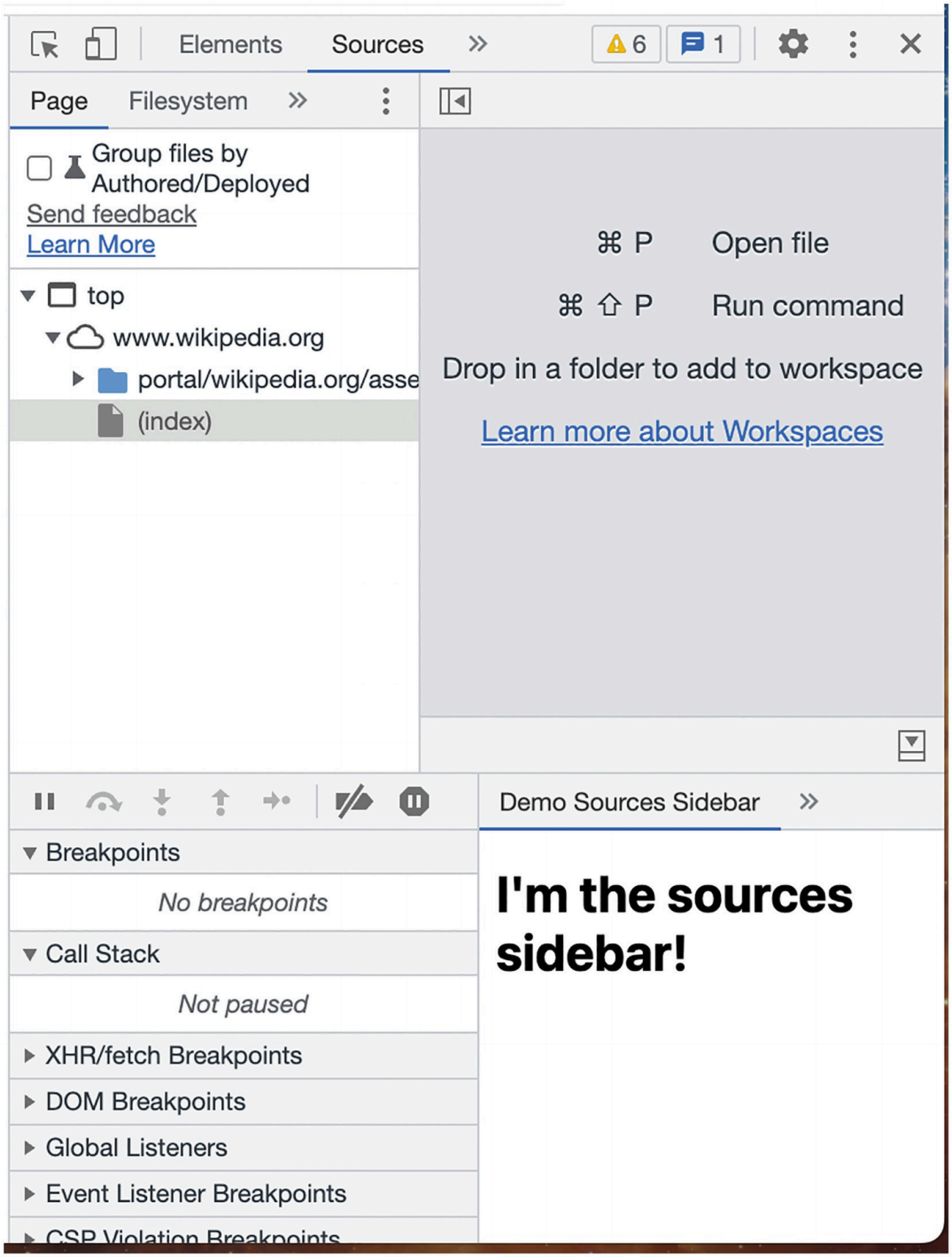

Figure 10-7 Viewing the new sources sidebar option

Click these options to view the new sidebar interfaces:

Of course, these views don't do anything useful yet. In the next section, we will cover the special methods these views can access.

> **Note** The panel and sidebar views will re-render the HTML page each time the tab is opened. For example, if you were to click into your custom panel, click over to the Elements tab, and click back over to the

original panel, it will have rendered two separate times. Keep this in mind should you need to preserve state.

The Devtools API

The Devtools API is an add-on to the WebExtensions API and only available to extension views inside the developer tools interface. The API contains the following four namespaces:

- `chrome.devtools.panels` is used to programmatically add custom panels and sidebars to the developer tools. These methods are typically called from the headless devtools page that is rendered when the developer tools interface is opened.
- `chrome.devtools.network` is used to sniff the network traffic in this page. It is similar to the `webNavigation` API, but the devtools version records traffic in the HTTP Archive format (HAR).
- `chrome.devtools.inspectedWindow` is used to inspect the current web page in ways that the normal WebExtensions API cannot. It can evaluate JavaScript expressions in the context of the page as well as view a list of resources (such as a document, script, or image) that the current web page is using.
- `chrome.devtools.recorder` is used to customize the developer tools Recorder panel. At the time this book was written, this API was in preview and only supported exporting recorder data.

NoteGoogle Chrome and other chromium-based browsers will fully support the Devtools API. Other browsers like Safari and Firefox may not support these APIs fully.

Sniffing network traffic

Most web developers will be very familiar with the developer tools Network panel. This panel shows extremely detailed real-time information about the active web page's outgoing network requests. A browser extension can tap into this same feed with the Devtools API.

The API allows you to access a log of recorded network requests, as well as add handlers for some network request lifecycle events.

The API will provide information about network requests in the HTTP Archive (HAR) format. A truncated example of the HAR format for a GET request to `wikipedia.org` is shown below:

Example HAR data for wikipedia.org GET request

```
{
  "_initiator": {
    "type": "other"
  },
  "_priority": "VeryHigh",
  "_resourceType": "document",
  "cache": {},
  "connection": "769999",
  "pageref": "page_2",
  "request": {
    "method": "GET",
    "url": "https://www.wikipedia.org/",
    "httpVersion": "http/2.0",
    "headers": [
      {
        "name": ":method",
        "value": "GET"
      },
      {
        "name": ":path",
        "value": "/"
      },
      {
        "name": ":scheme",
        "value": "https"
      },
      ...
    ],
    "queryString": [],
    "cookies": [
```

```
      ...
    ],
    "headersSize": -1,
    "bodySize": 0
  },
  "response": {
    "status": 304,
    "statusText": "",
    "httpVersion": "http/2.0",
    "headers": [
      {
        "name": "content-encoding",
        "value": "gzip"
      },
      {
        "name": "content-type",
        "value": "text/html"
      },
      ...
    ],
    "cookies": [],
    "content": {
      "size": 75189,
      "mimeType": "text/html"
    },
    "redirectURL": "",
    "headersSize": -1,
    "bodySize": 0,
    "_transferSize": 1145,
    "_error": null
  },
  "serverIPAddress": "208.80.154.224",
  "startedDateTime": "2022-08-24T15:28:35.006Z",
  "time": 45.61999998986721,
  "timings": {
    "blocked": 3.410000022381544,
    "dns": -1,
    "ssl": -1,
    "connect": -1,
    "send": 0.2370000000000001,
    "wait": 41.07399999056011,
```

```
    "receive": 0.8989999769255519,
    "_blocked_queueing": 1.8070000223815441
  }
}
```

For developers who have spent lots of time in the Network panel, you will quickly recognize this as the JSON data dump of all the information displayed in the panel.

The Devtools API only has one method and two events for you to use, but this is sufficient for you to be able to architect your own version of a network inspector. The following example creates a very simple custom network panel:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "devtools_page": "devtools.html"
}
```

Example 10-4a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <script src="devtools.js"></script>
  </body>
</html>
```

Example 10-4b devtools.html

```
chrome.devtools.panels.create("Devtools Traffic", "", "traffic_panel.html");
```

Example 10-4c devtools.js

```
<!DOCTYPE html>
<html>
  <head>
    <script src="traffic_panel.js" defer></script>
  </head>
```

```
  <body></body>
</html>
```

Example 10-4d traffic_panel.html

```
function logRequest(har) {
  document.body.innerHTML += `
  <div>
    ${har.request.method} ${har.request.url}
    (${har.response.status})
  </div>`;
}
// One-time call to acquire all the requests accumulated
// so far. This allows you to navigate between
// devtools panels without losing the traffic log.
chrome.devtools.network.getHAR((harLog) => {
  for (let har in harLog.entries) {
    logRequest(har);
  }
});
// Fires each time the top-level webpage changes URL
chrome.devtools.network.onNavigated.addListener((url) => {
  document.body.innerHTML += `<hr><h1>${url}</h1><hr>`;
});
// Fired each time anything in the webpage
// makes a network request
chrome.devtools.network.onRequestFinished.addListener(
  (har) => logRequest(har));
```

Example 10-4e traffic_panel.js

Load this extension, open the developer tools, and select the new "Devtools Traffic" panel. Navigate to any web page and you will see all the requests dump out into this panel, as shown in Figure **10-8**.

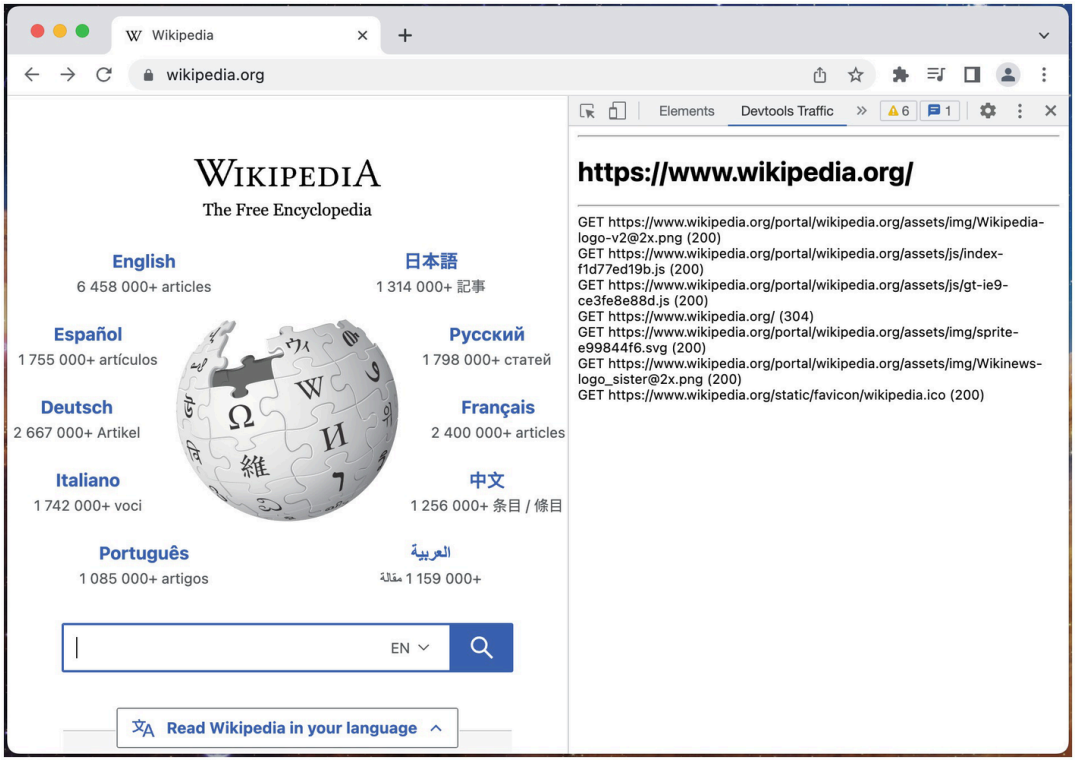

Figure 10-8 Custom network panel showing traffic coming from wikipedia.org

Inspecting the Page

The Devtools API allows you to run JavaScript expressions in the context of the web page using the `eval()` method. This method is very similar to `scripting.executeScript()`: you provide a piece of JavaScript in the local devtools context, it is passed up and evaluated in the remote web page context, and a serializable value can be returned. However, unlike the `scripting.executeScript()` method, `eval()` also provides access to the JavaScript state of the page. Additionally, `eval()` uses string expressions, whereas `executeScript` uses a function object or file reference.

In the *Content Scripts* chapter, we examined how a content script was unable to access the JavaScript state of the host page with an example showing how an injected script could not see the host page's global `jQuery` object. Using the Devtools API, it is now possible to see and interact with the host page's JavaScript state. This behavior is demonstrated in the following example:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "devtools_page": "devtools.html"
}
```

Example 10-5a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <script src="devtools.js"></script>
  </body>
</html>
```

Example 10-5b devtools.html

```
chrome.devtools.panels.create("Devtools Inspector", "", "inspect_panel.html");
```

Example 10-5c devtools.js

```
<!DOCTYPE html>
<html>
  <head>
    <script src="inspect_panel.js" defer></script>
  </head>
  <body>
    <button id="check">CHECK FOR JQUERY</button>
  </body>
</html>
```

Example 10-5d inspect_panel.html

```
document.querySelector("#check").addEventListener("click", () => {
  chrome.devtools.inspectedWindow.eval(
    `({
      'url': window.location.href,
      'usesJquery': !!window.jQuery
    })`,
```

```
      null,
      (result) => {
        const div = document.createElement("div");
        div.innerText = `${result.url} uses jQuery: ${result.usesJquery}`;
        document.body.appendChild(div);
      }
    );
});
```

Example 10-5e inspect_panel.js

Load this extension and open the devtools on `jquery.com`, which has a `jQuery` global object, and `wikipedia.org`, which does not (Figure 10-9). Clicking the button on each site will accurately record inside the devtools panel whether or not the site uses jQuery.

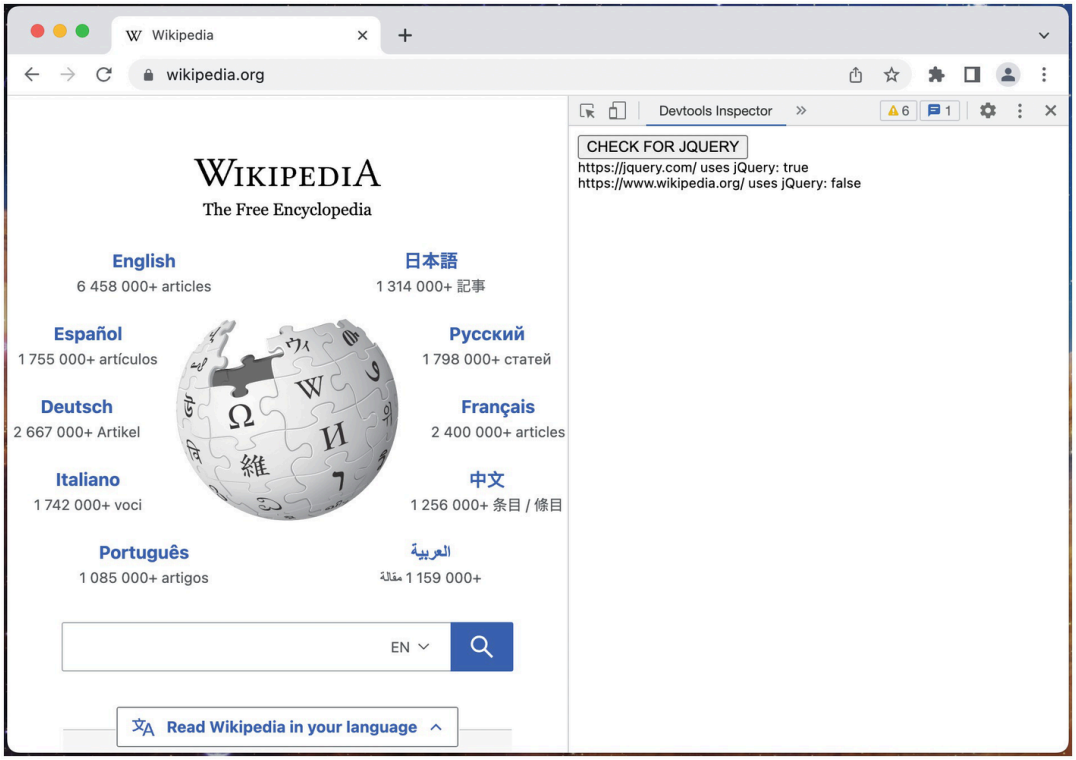

Figure 10-9 Inspecting the host page's JavaScript state

For complicated expressions, you can use an instantaneously invoked function expression (IIFE) that returns a value. The previous expression could be refactored to use an IIFE:

```
chrome.devtools.inspectedWindow.eval(
  `(() => {
```

```
      const url = window.location.href;
      const usesJquery = !!window.jQuery;
      return { url, usesJquery };
    })()`,
    ...
)
```

JavaScript expressions passed to `eval()` also have access to the browser's console API, so special console values and methods such as `$0` and `inspect()` can be used. These are especially useful when used in a sidebar that extends the Elements interface because direct integration with the Element panel's DOM tree explorer is possible. The following example does exactly this:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "devtools_page": "devtools.html"
}
```

Example 10-6a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <script src="devtools.js"></script>
  </body>
</html>
```

Example 10-6b devtools.html

```
chrome.devtools.panels.elements.createSidebarPane(
  "Devtools Inspector",
  (sidebar) => {
    sidebar.setPage("inspect_panel.html");
  }
);
```

Example 10-6c devtools.js

```
<!DOCTYPE html>
<html>
  <head>
    <script src="inspect_panel.js" defer></script>
  </head>
  <body>
    <button id="inspect">INSPECT IMG</button>
    <button id="tagname">ACTIVE TAG NAME</button>
  </body>
</html>
```

Example 10-6d inspect_panel.html

```
document.querySelector("#inspect").addEventListener("click", () => {
  chrome.devtools.inspectedWindow.eval(
    `inspect(document.querySelector('img'))`
  );
});
document.querySelector("#tagname").addEventListener("click",
  () => {
    chrome.devtools.inspectedWindow.eval(
      `$0?.tagName`,
      null,
      (result) => {
        const div = document.createElement("div");
        div.innerText = result;
        document.body.appendChild(div);
      });
  }
);
```

Example 10-6e inspect_panel.js

Load this extension, open the developer tools, select the Elements tab, and open the Devtools Inspector sidebar.

In this example, the "INSPECT IMG" button is calling `inspect()` on the first image to appear in the document. Because this is called from a sidebar inside the Elements panel, you will see this immediately reflected in the native browser's DOM inspector (Figure **10-10**).

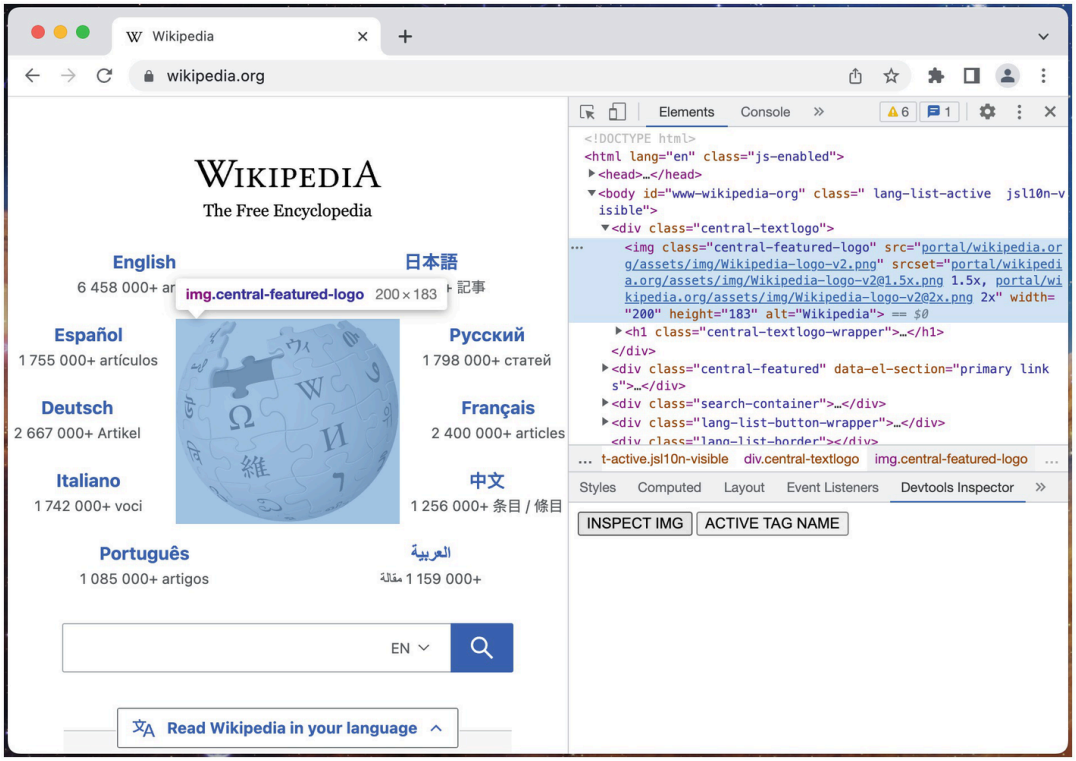

Figure 10-10 Programmatically inspecting the page's image

Clicking the ACTIVE TAG NAME button will use the `$0` token to reference whichever DOM inspector node was last inspected and print its tag name (Figure 10-11).

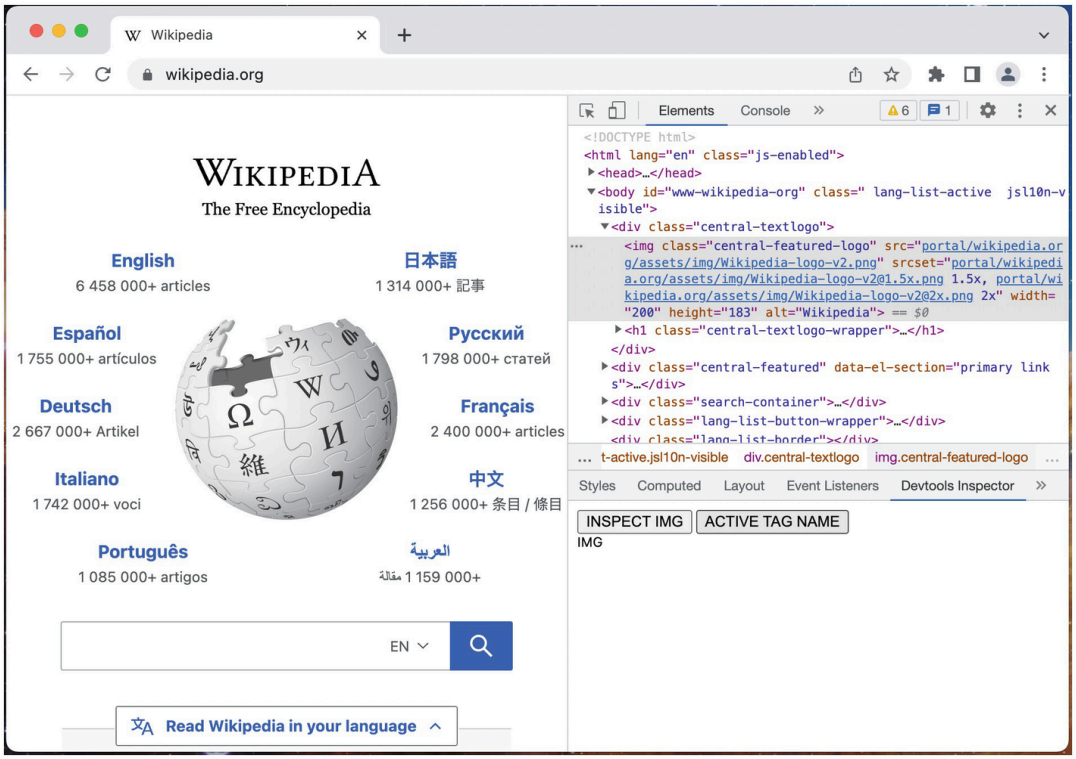

Figure 10-11 Accessing the selected DOM node's tag name

This bi-directional integration with the native browser allows you to build sophisticated devtools pages that can quickly direct the user around the page.

Content Scripts and Devtools Messaging

Typically, devtools pages are most useful when they can coordinate with content scripts injected into the page. Each component has its own responsibilities: the devtools pages are responsible for interacting with the Devtools API, the content scripts are responsible for managing page overlays and similar widgets, and the two can coordinate via messaging. The problems with this strategy are twofold:

- Although the Devtools API can access the host page's tab ID via `chrome.devtools.inspectedWindow.tabId`, it cannot use `chrome.tabs.sendMessage()` to dispatch a message to that specific tab.
- When sending messages from the content script to devtools, you must account for scenarios where there are multiple developer tools interfaces open simultaneously. Content scripts have no direct way of dispatching a message to only that page's devtools panel.

There are a handful of clever strategies to solve these issues:

- `runtime.sendMessage()` can solve the problem of the devtools page messaging the content script. The devtools page can include the tabID, and the content script can filter by the page's tab ID. The content script can then respond directly to that message, which will only be seen by the original devtools. To establish a pseudo-bidirectional channel, the content script can queue up messages, and the devtools page can intermittently poll.
- The devtools page establishes a long-lived connection with the background page. The background page manages a map of tab IDs to connections, so it can route each message to the correct connection.

- The devtools page combines an injected script with a content script that acts as an intermediary. It then uses `window.postMessage()` to pass messages to the content script.

Tip The Google Chrome team has an excellent writeup on this here: **https://developer.chrome.com/docs/extensions/mv3/devtools/#solutions**.

Other Devtools API Features

In addition to the examples shown in this chapter, the Devtools API includes some other interesting features:

- Devtools pages can evaluate `eval()` expressions as an extension content script using the `useContentScriptContext` option. This is useful to exchange data with the content script as well as access the WebExtensions API in the context of the page.
- Devtools pages are able to inspect which resources (documents, stylesheets, scripts, images, etc.) the page is currently using via `chrome.devtools.inspectedWindow.getResources()`, as well as listen for changes in those resources via the `onResourceAdded` and `onResourceContentCommitted` events.
- Devtools pages are able to force the page to reload with `chrome.devtools.inspectedWindow.reload()`. This `reload()` method also allows you to ignore the browser cache, inject custom scripts upon the next load, and spoof the user agent.
- Devtools pages can define a plugin to export Recorder data via `chrome.devtools.recorder.registerRecorderExtensionPlugin()`. For more information on the Recorder panel, see here: **https://developer.chrome.com/docs/devtools/recorder/**

Summary

In this chapter, you learned about how an extension can extend the browser's native developer tools interface. We went through the unusual way that an extension injects user interfaces into the developer

tools, as well as all the major features of the Devtools API that only devtools pages have access to.

In the next chapter, we will cover all the major WebExtensions APIs and how they can be used.